Aside - Pendo Analytics Fixes_19NOV25

🔴 CRITICAL - Must Complete Before Friday

1. SMS Metadata Implementation (Development)

Status: ❌ BLOCKED - Not deployed correctly, needs debugging
Owner: Development Team
Impact: Blocks 6 user segments, makes SMS-only users appear inactive

What this is: Add SMS activity data to Pendo visitor profiles so SMS-only users show as "active"

The Problem:

- SMS-only users appear "inactive" in Pendo because "Last Seen" only tracks web visits
- User who texts 50x/month but visits web once = shows as "inactive" ❌
- User who visits web daily but never texts = shows as "very active" ✅
- This gives inaccurate view of who's engaged

Implementation Steps:

1. Add `pendo.identify()` calls to SMS webhook to update visitor profiles with SMS metadata
2. Implement daily cron job to update `daysSinceLastSms` for all users
3. Run one-time backfill script to populate metadata for existing users
4. Test that visitor profiles update within 30 seconds of SMS send

Visitor Profile Fields to Add:

- `lastSmsDate`, `totalSmsMessages`, `smsMessagesLast7Days`, `smsMessagesLast30Days`
- `daysSinceLastSms`, `daysSinceLastWebVisit`
- `primaryPlatform` (sms/web/balanced), `smsActivityPercentage`
- `engagementLevel` (power_user/active/casual/dormant)
- `totalBoards`, `privateBoardCount`, `sharedBoardCount`, `hasSharedBoards`

Why Critical: Without this, can't create accurate "Active Users" segments that combine SMS + web activity

---

Implementation Instructions:

⚠️ CRITICAL FOR REPLIT: Verification Steps at the End
This has been attempted twice and not implemented correctly. Please follow the verification steps at the end to confirm the code is actually running.

STEP 1: Add Helper Functions

Create these helper functions (add to a utilities file or at the top of your SMS webhook file):

```
/**
 * Get SMS activity statistics for a phone number
 * @param {string} phoneNumber - User's phone number (E.164 format)
 * @returns {Object} SMS activity stats
 */
async function getSMSActivityStats(phoneNumber) {
  const now = new Date();
  const sevenDaysAgo = new Date(now - 7 * 24 * 60 * 60 * 1000);
  const thirtyDaysAgo = new Date(now - 30 * 24 * 60 * 60 * 1000);

  // Get message counts for different time periods
  const [totalMessages, last7Days, last30Days, lastMessage] = await Promise.all([
    db.messages.count({
      where: { from: phoneNumber }
    }),
    db.messages.count({
      where: {
        from: phoneNumber,
        createdAt: { gte: sevenDaysAgo }
      }
    }),
    db.messages.count({
      where: {
        from: phoneNumber,
        createdAt: { gte: thirtyDaysAgo }
      }
    }),
    db.messages.findOne({
      where: { from: phoneNumber },
      order: [['createdAt', 'DESC']]
    })
  ]);
```

```
  // Calculate platform preference
  const webVisits = await db.webSessions.count({
    where: {
      phone: phoneNumber,
      createdAt: { gte: thirtyDaysAgo }
    }
  });

  const totalActivity = last30Days + webVisits;
  const smsPercentage = totalActivity > 0
    ? Math.round((last30Days / totalActivity) * 100)
    : 100;

  let primaryPlatform;
  if (smsPercentage > 70) primaryPlatform = 'sms';
  else if (smsPercentage < 30) primaryPlatform = 'web';
  else primaryPlatform = 'balanced';

  return {
    totalMessages,
    last7Days,
    last30Days,
    lastMessageDate: lastMessage ? lastMessage.createdAt : null,
    smsPercentage,
    primaryPlatform
  };
}

/**
 * Calculate user engagement level based on activity
 * @param {number} smsLast30Days - SMS messages in last 30 days
 * @param {number} daysSinceLastWebVisit - Days since last web visit
 * @returns {string} Engagement level
 */
function calculateEngagementLevel(smsLast30Days, daysSinceLastWebVisit) {
  // Consider both SMS and web activity
  const recentWebActivity = daysSinceLastWebVisit !== null && daysSinceLastWebVisit < 30;

  if (smsLast30Days >= 10 || (smsLast30Days >= 5 && recentWebActivity)) {
    return 'power_user';
  } else if (smsLast30Days >= 3 || recentWebActivity) {
    return 'active';
  } else if (smsLast30Days >= 1) {
    return 'casual';
```

```
  } else {
    return 'dormant';
  }
}
```

STEP 2: Update Twilio SMS Webhook

Find your Twilio webhook handler (the route that receives SMS messages). It probably looks like:

```
app.post('/api/webhook/twilio', async (req, res) => {
  const { From, Body, MessageSid } = req.body;
  // ... your existing code ...
});
```

ADD THIS CODE after your existing `pendo.track('SMS Message Received')` call:

```
app.post('/api/webhook/twilio', async (req, res) => {
  const { From, Body, MessageSid } = req.body;

  try {
    // ... your existing SMS processing logic ...
    // ... your existing pendo.track('SMS Message Received') ...

    // ========== NEW CODE STARTS HERE ==========

    // Get SMS activity stats
    const smsStats = await getSMSActivityStats(From);

    // Get user info
    const user = await db.users.findOne({ where: { phone: From } });

    if (user) {
      // Calculate days since signup
      const daysSinceSignup = Math.floor(
        (Date.now() - new Date(user.createdAt).getTime()) / (1000 * 60 * 60 * 24)
      );

      // Calculate days since last web visit
      const lastWebSession = await db.webSessions.findOne({
        where: { phone: From },
        order: [['createdAt', 'DESC']]
      });
```

```
    const daysSinceLastWebVisit = lastWebSession
      ? Math.floor((Date.now() - new Date(lastWebSession.createdAt).getTime()) / (1000 * 60 *
60 * 24))
      : null;

    // Get board counts
    const [privateBoards, sharedBoards] = await Promise.all([
      db.boards.count({
        where: { owner_phone: From, is_shared: false }
      }),
      db.boardMembers.count({
        where: { member_phone: From }
      })
    ]);

    // Update Pendo visitor profile with SMS metadata
    await pendo.identify({
      visitorId: From,
      accountId: 'aside',
      visitor: {
        // Basic user info
        phone: From,
        firstName: user.firstName || null,
        lastName: user.lastName || null,
        email: user.email || null,

        // Signup info
        signupDate: user.createdAt.toISOString(),
        signupMethod: user.signupMethod || 'sms',
        daysSinceSignup: daysSinceSignup,

        // SMS Activity Metadata (CRITICAL FOR SEGMENTATION)
        lastSmsDate: new Date().toISOString(),
        totalSmsMessages: smsStats.totalMessages,
        smsMessagesLast7Days: smsStats.last7Days,
        smsMessagesLast30Days: smsStats.last30Days,
        daysSinceLastSms: 0, // Just sent a message!

        // Web Activity Metadata
        daysSinceLastWebVisit: daysSinceLastWebVisit,

        // Platform Preference
        primaryPlatform: smsStats.primaryPlatform,
```

```
        smsActivityPercentage: smsStats.smsPercentage,

        // User Engagement Level
        engagementLevel: calculateEngagementLevel(smsStats.last30Days,
daysSinceLastWebVisit),

        // Content Stats
        totalBoards: privateBoards + sharedBoards,
        privateBoardCount: privateBoards,
        sharedBoardCount: sharedBoards,
        hasSharedBoards: sharedBoards > 0,

        // Feature Adoption
        hasCreatedBoard: privateBoards > 0,
        hasJoinedSharedBoard: sharedBoards > 0,

        // Last updated
        profileLastUpdated: new Date().toISOString()
      }
    });

    // Log success for verification
    console.log(`✅ SMS metadata updated for ${From}`);
  }

  // ========== NEW CODE ENDS HERE ==========

  res.sendStatus(200);
 } catch (error) {
  console.error('Error processing SMS:', error);
  res.sendStatus(500);
 }
});
```

STEP 3: Create Daily Cron Job

Add this code to set up a daily cron job that updates `daysSinceLastSms` for all users:

```
const cron = require('node-cron');

/**
 * Daily cron job to update daysSinceLastSms for all users
 * Run this once per day at midnight
 */
```

```
async function updateDaysSinceLastSms() {
  console.log('🔄 Starting daily SMS activity update...');

  const allUsers = await db.users.findAll({
    attributes: ['phone', 'createdAt']
  });

  let successCount = 0;
  let errorCount = 0;

  for (const user of allUsers) {
    try {
      // Get last SMS message
      const lastMessage = await db.messages.findOne({
        where: { from: user.phone },
        order: [['createdAt', 'DESC']]
      });

      if (lastMessage) {
        const daysSinceLastSms = Math.floor(
          (Date.now() - new Date(lastMessage.createdAt).getTime()) / (1000 * 60 * 60 * 24)
        );

        // Update Pendo profile
        await pendo.identify({
          visitorId: user.phone,
          visitor: {
            daysSinceLastSms: daysSinceLastSms,
            profileLastUpdated: new Date().toISOString()
          }
        });

        successCount++;
      }
    } catch (error) {
      console.error(`❌ Error updating SMS days for ${user.phone}:`, error);
      errorCount++;
    }
  }

  console.log(`✅ Daily SMS update complete: ${successCount} success, ${errorCount} errors`);
}

// Schedule cron job to run every day at midnight
```

```
cron.schedule('0 0 * * *', () => {
  updateDaysSinceLastSms();
});

// Also export so it can be manually triggered for testing
module.exports = { updateDaysSinceLastSms };
```

STEP 4: One-Time Backfill Script

Create a separate script file (e.g., `backfill-sms-metadata.js`) to backfill existing users:

```
/**
 * One-time script to backfill SMS metadata for existing users
 * Run this ONCE after deploying the pendo.identify() updates
 */
async function backfillSMSMetadata() {
  console.log('🚀 Starting SMS metadata backfill...');

  const allUsers = await db.users.findAll();
  let processed = 0;

  for (const user of allUsers) {
    try {
      // Get SMS stats
      const smsStats = await getSMSActivityStats(user.phone);

      // Calculate tenure
      const daysSinceSignup = Math.floor(
        (Date.now() - new Date(user.createdAt).getTime()) / (1000 * 60 * 60 * 24)
      );

      // Calculate days since last SMS
      const lastMessage = await db.messages.findOne({
        where: { from: user.phone },
        order: [['createdAt', 'DESC']]
      });

      const daysSinceLastSms = lastMessage
        ? Math.floor((Date.now() - new Date(lastMessage.createdAt).getTime()) / (1000 * 60 * 60 *
24))
        : null;

      // Get web visit info
      const lastWebSession = await db.webSessions.findOne({
```

```
      where: { phone: user.phone },
      order: [['createdAt', 'DESC']]
    });

    const daysSinceLastWebVisit = lastWebSession
      ? Math.floor((Date.now() - new Date(lastWebSession.createdAt).getTime()) / (1000 * 60 *
60 * 24))
      : null;

    // Get board counts
    const [privateBoards, sharedBoards] = await Promise.all([
      db.boards.count({
        where: { owner_phone: user.phone, is_shared: false }
      }),
      db.boardMembers.count({
        where: { member_phone: user.phone }
      })
    ]);

    // Update Pendo
    await pendo.identify({
      visitorId: user.phone,
      accountId: 'aside',
      visitor: {
        phone: user.phone,
        firstName: user.firstName || null,
        lastName: user.lastName || null,
        signupDate: user.createdAt.toISOString(),
        signupMethod: user.signupMethod || 'sms',
        daysSinceSignup: daysSinceSignup,
        lastSmsDate: lastMessage ? lastMessage.createdAt.toISOString() : null,
        totalSmsMessages: smsStats.totalMessages,
        smsMessagesLast7Days: smsStats.last7Days,
        smsMessagesLast30Days: smsStats.last30Days,
        daysSinceLastSms: daysSinceLastSms,
        daysSinceLastWebVisit: daysSinceLastWebVisit,
        primaryPlatform: smsStats.primaryPlatform,
        smsActivityPercentage: smsStats.smsPercentage,
        engagementLevel: calculateEngagementLevel(smsStats.last30Days,
daysSinceLastWebVisit),
        totalBoards: privateBoards + sharedBoards,
        privateBoardCount: privateBoards,
        sharedBoardCount: sharedBoards,
        hasSharedBoards: sharedBoards > 0,
```

```
        hasCreatedBoard: privateBoards > 0,
        hasJoinedSharedBoard: sharedBoards > 0,
        profileLastUpdated: new Date().toISOString()
      }
    });

    processed++;

    if (processed % 10 === 0) {
      console.log(`📊 Processed ${processed}/${allUsers.length} users...`);
    }

    // Rate limit to avoid overwhelming Pendo API
    await new Promise(resolve => setTimeout(resolve, 100));

  } catch (error) {
    console.error(`❌ Error backfilling ${user.phone}:`, error);
  }
 }

 console.log(`✅ Backfill complete! Processed ${processed} users.`);
}

// Run the backfill
backfillSMSMetadata();
```

🔍 VERIFICATION STEPS FOR REPLIT

These steps are CRITICAL. Previous implementations failed because verification wasn't done.

Test 1: Check that helper functions exist

```
// Add this temporary test endpoint to verify functions are defined:
app.get('/test/sms-metadata-functions', async (req, res) => {
 const tests = {
  getSMSActivityStats: typeof getSMSActivityStats === 'function',
  calculateEngagementLevel: typeof calculateEngagementLevel === 'function',
  updateDaysSinceLastSms: typeof updateDaysSinceLastSms === 'function'
 };

 res.json({
  message: 'Function check',
```

```
    tests: tests,
    allPass: Object.values(tests).every(t => t === true)
  });
});
```

Expected result:

```
{
  "message": "Function check",
  "tests": {
    "getSMSActivityStats": true,
    "calculateEngagementLevel": true,
    "updateDaysSinceLastSms": true
  },
  "allPass": true
}
```

If any show `false`, the functions were not added correctly.

Test 2: Verify pendo.identify() is being called

Add console logging to verify the code runs:

```
// In your SMS webhook, right before the pendo.identify() call, add:
console.log('🔵 About to call pendo.identify() with SMS metadata for:', From);

await pendo.identify({
  // ... all your visitor data ...
});

console.log('✅ pendo.identify() completed successfully for:', From);
```

Expected result in logs:

```
🔵 About to call pendo.identify() with SMS metadata for: +16155551234
✅ pendo.identify() completed successfully for: +16155551234
```

If you don't see these logs after sending an SMS, the code is not running.

Test 3: Send test SMS and check Pendo

1. **Before test:** Note the timestamp

2. **Action:** Send an SMS to Aside from a test number (e.g., "+16155551234")

3. **Wait:** 30 seconds

4. **Check Pendo:**

    - Go to: https://app.pendo.io
    - Navigate to: People → Visitors
    - Search for: the test phone number
    - Click on the visitor profile
5. **Verify these fields exist and have recent timestamps:**

    - `lastSmsDate` - Should be within last minute
    - `totalSmsMessages` - Should be > 0
    - `smsMessagesLast7Days` - Should be > 0
    - `daysSinceLastSms` - Should be 0
    - `primaryPlatform` - Should be "sms", "web", or "balanced"
    - `engagementLevel` - Should be one of: "power_user", "active", "casual", "dormant"
    - `profileLastUpdated` - Should be within last minute

If these fields don't exist or have old timestamps, the pendo.identify() call is not working.

Test 4: Verify cron job is scheduled

Add this test endpoint:

```
app.get('/test/cron-status', (req, res) => {
  res.json({
    message: 'Cron job status',
    isScheduled: typeof cron !== 'undefined' && cron.getTasks ? cron.getTasks().size > 0 : false,
    note: 'Cron job should run daily at midnight'
  });
});
```

Expected result:

```
{
  "message": "Cron job status",
  "isScheduled": true,
  "note": "Cron job should run daily at midnight"
```

```
}
```

Test 5: Manually trigger cron job

Add this test endpoint to manually run the daily update:

```
app.post('/test/trigger-daily-update', async (req, res) => {
  try {
    console.log('🔵 Manually triggering daily SMS update...');
    await updateDaysSinceLastSms();
    res.json({ success: true, message: 'Daily update completed' });
  } catch (error) {
    res.status(500).json({ success: false, error: error.message });
  }
});
```

Test it:

POST /test/trigger-daily-update

Expected result:

- Console shows: "✅ Daily SMS update complete: X success, 0 errors"
- Response: `{"success": true, "message": "Daily update completed"}`

Test 6: Run backfill and verify

1. **Run the backfill script once:**

```
node backfill-sms-metadata.js
```

2. **Expected console output:**

```
🚀 Starting SMS metadata backfill...
📊 Processed 10/45 users...
📊 Processed 20/45 users...
✅ Backfill complete! Processed 45 users.
```

3. **Check 3 random users in Pendo:**
    - Go to: People → Visitors
    - Search for 3 different phone numbers

- Verify ALL metadata fields are populated
- Verify `totalSmsMessages` matches actual message count in database

If users don't have metadata after backfill, the script didn't run correctly.

🚨 COMMON FAILURES & HOW TO FIX

Problem 1: "pendo is not defined"

- **Cause:** Pendo SDK not imported
- **Fix:** Add `const pendo = require('pendo');` at top of file

Problem 2: "db is not defined"

- **Cause:** Database not imported
- **Fix:** Add your database import (adjust to match your setup)

Problem 3: Functions not found

- **Cause:** Helper functions not in scope
- **Fix:** Ensure helper functions are defined BEFORE the webhook handler

Problem 4: No fields in Pendo

- **Cause:** pendo.identify() call not executing
- **Fix:** Check console logs for errors, verify pendo.identify() is actually being called

Problem 5: Cron job not running

- **Cause:** node-cron not installed or not scheduled correctly
- **Fix:** Run `npm install node-cron`, verify cron.schedule() is called

✅ SUCCESS CRITERIA

All of these must be TRUE:

1. ✅ GET /test/sms-metadata-functions returns `"allPass": true`
2. ✅ Console shows "🔵 About to call pendo.identify()" when SMS received
3. ✅ Console shows "✅ pendo.identify() completed successfully" after each SMS
4. ✅ Test SMS results in updated visitor profile in Pendo within 30 seconds
5. ✅ All metadata fields visible in Pendo visitor profile
6. ✅ GET /test/cron-status returns `"isScheduled": true`

7. ✅ POST /test/trigger-daily-update completes without errors
8. ✅ Backfill script processes all users and shows success message
9. ✅ Random spot-check of 3 users shows complete metadata in Pendo

If ANY of these fail, the implementation is NOT complete.

---

2. Add Missing Event Properties (Development)

Status: ❌ TODO - Development needed
Owner: Development Team
Impact: Enables behavioral analysis, content type tracking, time pattern analysis

What this is: Add properties to existing Pendo events to make them more useful for filtering and analysis

Properties to ADD:

For "SMS Message Received" event (5 properties):

- `content_type` (string: "link", "text_note", "idea_note")
- `contains_url` (boolean: true/false)
- `is_new_hashtag` (boolean: did this message create a new board?)
- `day_of_week` (string: "Monday", "Tuesday", etc.)
- `time_of_day` (string: "morning", "afternoon", "evening", "night")

For "Content Added via SMS" event (1 property):

- `is_new_board` (boolean: first time using this hashtag?)

For "New Board Created via SMS" event (2 properties):

- `first_message_content_type` (string: "link", "text_note", "idea_note")
- `user_tenure_days` (number: days since user signed up)

Total: 8 new properties across 3 events

Why Important: Enables filtering events by content type, understanding time patterns, tracking board creation triggers

Implementation Instructions:

Where to add these properties: These properties should be added to your existing `pendo.track()` calls in the Twilio SMS webhook handler (where you already track "SMS Message Received", "Content Added via SMS", and "New Board Created via SMS").

Step 1: Add helper functions

```
/**
 * Detect content type from message text
 */
function detectContentType(messageText) {
  const hasUrl = /https?:\/\//.test(messageText);

  if (hasUrl) {
    return 'link';
  }

  // Check for idea indicators
  const ideaIndicators = ['idea:', 'thought:', 'remember to', 'don\'t forget'];
  const lowerText = messageText.toLowerCase();
  const hasIdeaIndicator = ideaIndicators.some(indicator => lowerText.includes(indicator));

  if (hasIdeaIndicator) {
    return 'idea_note';
  }

  return 'text_note';
}

/**
 * Get day of week from timestamp
 */
function getDayOfWeek(timestamp = new Date()) {
  const days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday'];
  return days[timestamp.getDay()];
}

/**
 * Get time of day from timestamp
 */
function getTimeOfDay(timestamp = new Date()) {
  const hour = timestamp.getHours();

  if (hour >= 5 && hour < 12) return 'morning';
  if (hour >= 12 && hour < 17) return 'afternoon';
  if (hour >= 17 && hour < 21) return 'evening';
```

```
  return 'night';
}

/**
 * Check if hashtag already exists for user
 */
async function isNewHashtag(phoneNumber, hashtag) {
  const existingBoard = await db.boards.findOne({
    where: {
      owner_phone: phoneNumber,
      name: hashtag
    }
  });
  return !existingBoard;
}

/**
 * Calculate user tenure in days
 */
async function getUserTenureDays(phoneNumber) {
  const user = await db.users.findOne({
    where: { phone: phoneNumber }
  });

  if (!user) return 0;

  return Math.floor(
    (Date.now() - new Date(user.createdAt).getTime()) / (1000 * 60 * 60 * 24)
  );
}
```

Step 2: Update "SMS Message Received" event

```
// In your Twilio webhook where you currently track "SMS Message Received"
// FIND this existing code:
await pendo.track('SMS Message Received', {
  user_id: From,
  // ... your existing properties ...
});

// UPDATE to this:
const messageTimestamp = new Date();
const contentType = detectContentType(Body);
const containsUrl = /https?:\/\//.test(Body);
```

```
// Check if any hashtags are new (for first hashtag only, or you can loop through all)
const hashtags = Body.match(/#\w+/g) || [];
const firstHashtag = hashtags[0];
const isNew = firstHashtag ? await isNewHashtag(From, firstHashtag) : false;

await pendo.track('SMS Message Received', {
  user_id: From,
  // ... your existing properties ...

  // NEW PROPERTIES:
  content_type: contentType,
  contains_url: containsUrl,
  is_new_hashtag: isNew,
  day_of_week: getDayOfWeek(messageTimestamp),
  time_of_day: getTimeOfDay(messageTimestamp)
});
```

Step 3: Update "Content Added via SMS" event

```
// In your code where you track "Content Added via SMS"
// FIND this existing code:
await pendo.track('Content Added via SMS', {
  user_id: From,
  boardName: boardName,
  // ... your existing properties ...
});

// UPDATE to this:
const isNewBoard = await isNewHashtag(From, boardName);

await pendo.track('Content Added via SMS', {
  user_id: From,
  boardName: boardName,
  // ... your existing properties ...

  // NEW PROPERTY:
  is_new_board: isNewBoard
});
```

Step 4: Update "New Board Created via SMS" event

```
// In your code where you track "New Board Created via SMS"
// FIND this existing code:
```

```
await pendo.track('New Board Created via SMS', {
  user_id: From,
  boardName: boardName,
  // ... your existing properties ...
});

// UPDATE to this:
const contentType = detectContentType(Body);
const tenureDays = await getUserTenureDays(From);

await pendo.track('New Board Created via SMS', {
  user_id: From,
  boardName: boardName,
  // ... your existing properties ...

  // NEW PROPERTIES:
  first_message_content_type: contentType,
  user_tenure_days: tenureDays
});
```

Step 5: Test the implementation

1. Send a test SMS with a link and new hashtag
2. Check Pendo → Data → Track Events → "SMS Message Received"
3. Verify you see the new properties with correct values:
    - content_type: "link"
    - contains_url: true
    - is_new_hashtag: true
    - day_of_week: (current day)
    - time_of_day: (current time period)

Note: If you need help finding where these events are currently tracked, search your codebase for `pendo.track('SMS Message Received')`

---

3. Add New Event (Development)

Status: ❌ TODO
Owner: Development Team
Impact: Track onboarding completion

Event to CREATE:

Welcome Message Sent - Fires when new user receives onboarding message

Properties needed:

- `user_signup_method` (sms/web/invite)
- `message_delivered` (boolean)
- `user_id`
- `timestamp`

Why Critical: Confirms new users receive onboarding message

Implementation Instructions:

Where to add this event: In your code where you send the welcome message to new users (likely when a new user signs up or completes verification).

Step 1: Add the tracking call

```
// In your function that sends the welcome SMS to new users
// After you successfully send the welcome message via Twilio:

async function sendWelcomeMessage(phoneNumber, signupMethod) {
  try {
    // Your existing code to send SMS via Twilio
    const message = await twilioClient.messages.create({
      body: `Welcome to Aside! 🎉\n\nI'm going to save everything you text me. Use #hashtags to organize.\n\nTry it - send me something!`,
      to: phoneNumber,
      from: ASIDE_PHONE_NUMBER
    });

    // NEW: Track that welcome message was sent
    await pendo.track('Welcome Message Sent', {
      user_id: phoneNumber,
      user_signup_method: signupMethod, // "sms", "web", or "invite"
      message_delivered: true, // Twilio successfully sent it
      timestamp: new Date().toISOString()
    });

    return message;

  } catch (error) {
    console.error('Error sending welcome message:', error);

    // Track failed delivery
```

```
    await pendo.track('Welcome Message Sent', {
      user_id: phoneNumber,
      user_signup_method: signupMethod,
      message_delivered: false, // Failed to send
      timestamp: new Date().toISOString(),
      error_message: error.message
    });

    throw error;
  }
}
```

Step 2: Determine where this function is called

The welcome message is likely sent in one of these places:

1. After new user signs up via SMS (texts "START")
2. After user completes phone verification on web
3. After user accepts an invite

Example integration:

```
// In your user signup/verification handler
app.post('/api/auth/verify', async (req, res) => {
  const { phoneNumber, verificationCode } = req.body;

  try {
    // Your existing verification logic
    const verified = await verifyCode(phoneNumber, verificationCode);

    if (verified) {
      // Check if this is a new user
      const isNewUser = await isFirstTimeUser(phoneNumber);

      if (isNewUser) {
        // Create user account
        await createUser(phoneNumber);

        // Send welcome message and track it
        await sendWelcomeMessage(phoneNumber, 'web'); // or 'sms' or 'invite' depending on
signup method
      }

      res.json({ success: true });
```

```
    }
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Step 3: Test the implementation

1. **Test Case 1: New SMS signup**
    - Text "START" to Aside from a new number
    - Verify you receive welcome message
    - Check Pendo → Data → Track Events → "Welcome Message Sent"
    - Verify properties: user_signup_method: "sms", message_delivered: true
2. **Test Case 2: New web signup**
    - Sign up via web with new phone number
    - Complete verification
    - Verify you receive welcome message
    - Check Pendo for event with user_signup_method: "web"
3. **Test Case 3: Invite signup**
    - Accept an invite with new phone number
    - Verify welcome message and Pendo event with user_signup_method: "invite"

Step 4: Verify event appears in Pendo

After testing, go to Pendo:

- Navigate to: Data → Track Events
- Search for: "Welcome Message Sent"
- Click on the event
- Verify you see events with all the properties populated correctly